

Predictive Analytics on Stock Prices, Retail Sales, and Restaurant Tips

1. Introduction

This report contains a full predictive analytics project for three different types of problems: stock price prediction with an LSTM neural network, sales forecasting with the Holt-Winters Exponential Smoothing algorithm, and waiter tips regression with various linear and non-linear regression models. In addition to these three modeling exercises, we used joblib to package the best tips model into a deployable prediction pipeline, showing how to get from analysis to production. All the data throughout this project is made up and is generated programmatically to create realistic looking trends, seasonality and random noise in each of the three domains. The aim is not just to get the models to run, but to find out why some models work well, how to fairly test them on new data, and how to interpret the results and what they imply for real-life business decisions.

2. Methodology

2.1 Dataset Generation

The data set was created by using three different worksheets that correspond to the three domains of the problems. The Stock Prices sheet has 750 trading days of OHLC data and volume for stock from a made-up company called INNOVATETECH. The Sales Data sheet has 730 days of daily sales data for 4 stores in a fictional retail chain called BazaarMart; Downtown, Uptown, Suburb, and Mall. The Waiter Tips sheet consists of 300 restaurant bills from a made-up restaurant called SpiceGarden, along with the total bill, the tip, the sex of the guests, whether or not they smoked, the day of the week, the time of day, and the number of guests in the party. The stock prices were drawn from a random walk with a slight positive drift, the sales were created by adding to the random walk a linear trend, a weekly seasonality (weekends are about 25 per cent higher in sales), and a yearly seasonality (November and December sales are about 20 per cent higher than the year before). There was a high degree of correlation between the tips and the bill amount – about 17% – plus some random noise added in.

2.2 Stock Price Prediction with LSTM

The stock task consisted of INNOVATETECH, and we utilized merely its closing costs. The inputs were first scaled between 0 and 1 with a MinMaxScaler since inputs to a neural network are best trained when they are on a small uniform scale. We then created 20 day sliding windows and for each window used the window to forecast the following day's closing price. The LSTM network itself was made small, with 32 units in the LSTM layer, a dense layer of 16 units with ReLU activation, and a single output neuron which predicts the price for the next day. The model was compiled with the Adam optimizer and MSE loss and trained for 15 epochs with a batch size of 16. Importantly, the data has been divided by date — the first 80% of the data timeline was used for training, and the last 20% for testing. It is crucial for time-series data, and why it is explained in Section 4.

2.3 Sales Forecasting with Holt-Winters

The Downtown store was chosen for the sales task. The daily sales series was decomposed into trend, weekly seasonal and residual components by applying `seasonal_decompose`. Next, we

fit a Holt-Winters Exponential Smoothing model with an additive trend and an additive weekly seasonality, where the seasonal period corresponds to 7, which is the weekly cycle. The last 60 days were reserved as the test set, to allow us to verify the model's forecasting error using data not previously encountered.

2.4 Waiter Tips Regression

In the tips task, we have created a complete preprocessing pipeline, one-hot-encoding the categorical features, sex, smoker, day, time, and standardizing the numerical features, total_bill, size. This pipeline was then used for training four models. The first one was taking a very basic approach with just total_bill and a basic linear regression. The other three models were a linear regression with all features, a Ridge regression with the regularization strength of 1.0, and a Random Forest with 200 trees. The train/test split was randomly done with a fixed random state, which is suitable here since the number of tips does not have a meaningful time order.

3. Results

3.1 Stock Price Prediction

The LSTM had an RMSE of \$4.51 on the held-out test period. This means that on average, its next-day price prediction was about four and a half dollars away from the true closing price. With this synthetic series' daily volatility averaging approximately 1.7 percent, or two to three dollars in absolute price movement, the model successfully experienced the overall price level, though not all of the short-term price movements. This is a realistic and expected result on a simple LSTM that takes only the last 20 days of closing price data.

3.2 Sales Forecasting

Holt-Winters gave a RMSE of 137.5 units per day for the Downtown store for 60 days of testing. This means that Downtown has an average weekly sales rate of about 20-25% relative error, which is quite good considering the amount of random noise we added to the datasets to create. The weekly seasonality and the general trend of the data were well captured by the model, but it was not able to predict the day-to-day random fluctuation.

3.3 Waiter Tips

The results of the tips comparison were interesting and somewhat paradoxical. The simple baseline with only total_bill had the highest R^2 value of 0.826 followed by Ridge and the refined Linear Regression at 0.821 and Random Forest at 0.775 respectively. That is, the simplest model won. This is because the correlation between total_bill and tip in our data is very high, approximately 0.918, meaning that the majority of the variation in the tip is attributable to the variation in the bill. Incorporating features like sex, smoker, day and time added more noise than signal and Random Forest, which is known to overfit on small tabular datasets, actually performed the worst. This is an important lesson, one that is very realistic: complexity is not always a virtue and should be validated based on data and not assumption.

3.4 Deployment

The whole pipeline (the preprocessing and the model) was saved to tips_pipeline.joblib. We then simulated a new session and made predictions on two new bills for which the file was loaded. The model estimated a tip of \$7.59 for a \$45.00 dinner bill, for four people. It said it

would predict a \$3.24 lunch cost for two people at a \$18.50 bill. These values make sense and are consistent with the training provided data. The pipeline does not require any retraining to be dropped into any Python application or Web API.

4. Why Time-Based Splits matter

Deliberate lack of random train/test split was used for the stock and sales models. Rather, we used previous data for training, and the following data were used for testing. This is because if it were split randomly, future information would seep into the training set. If the price for December 2025 was in the training set, a model would have been learned from the future price data and then be tested in June 2024 which is an improbable scenario. During actual deployments, we will always make predictions about the future, based on the past, and the time-based split will faithfully emulate that. If that's what we did, we would have had the accuracy we saw on paper and failed as soon as we saw real future data that we hadn't seen before.

5. Limitations

Several caveats for this project are pointed out. Firstly, the dataset is synthetic, meaning that the models are trained with patterns which are much purer than the patterns in real life. These models would not be able to cope with the structural breaks, regime changes and chaotic jumps present in real stock markets and retail sales. Secondly, there is a training history that is limited. Seven hundred and fifty days is just about 3 years, a very limited seasonal cycle for the sales series. Third, the LSTM only takes the closing price, and not volume, moving averages or outside influences like news, earnings, etc., which would have increased its predictive power. Fourth, the Random Forest was overfitted due to the small number of samples (300) in the dataset and the relatively small number of features (six). Fifth, the tips data set is small, with 300 rows of data, which increases the confidence interval for any prediction.

6. Improvement Ideas

The most promising approach for the stock model is to include technical indicators like 5-day and 20-day moving averages, RSI, and trading volume, then experiment with longer history training, a stacked LSTM model, or a Transformer model. Similarly, in the sales model, changing to SARIMA or Facebook Prophet would enable the model to automatically account for multiple seasonality and use holiday calendars. It's possible that separate models would be more accurate than training them together because of such a large number of stores. For the tips model, the most significant improvements would be featuring engineering (e.g. calculate the tip as a percentage of bill and target that, as the tip is the target), use more data generally, and consider gradient boosting techniques (e.g. XGBoost), with appropriate cross validation.

7. Deployment Strategy for Stock and Sales

Joblib was used to deploy the tips pipeline, and the same pattern can be used to deploy the stock and sales models, although there are important differences. Firstly, the stock LSTM should be saved with the MinMaxScaler, and a scheduled batch job should be created to retrieve the last 20 days of prices from an API, scale the prices using the saved MinMaxScaler, make prediction of the next day's price using the saved stock LSTM, and save the prediction into the database. The Holt-Winters sales model should be re-fitted daily on the last two years of sales

and then make 7-day forecasts that should be input into a planning system. These are both batch jobs and not an on-demand API, as they rely on the most up-to-date history. The tips model, on the other hand, is truly real-time, because each request you receive has the bill information inside of it, and the model returns a prediction immediately. This contrast brings a difference in the deployment — time-series models usually require a batch inference and retraining periodically, whereas tabular models can be served on demand.

8. Business Implications

This project has several business implications. In a stock trading company, a RMSE of \$4.51 for a stock price range of \$150 to \$250 would indicate that the model's prediction would be within approximately two percent of the actual price which can be valuable in determining the direction of the stock and in recognizing extreme days. For retail management, if the store in question sells 500-700 per day, then the RMSE of 137.5 units per day means that the store manager has a forecasting tool with a relative error of about 20-25 percent, sufficient for planning the inventory but require safety stock buffers. A simple linear model provides the restaurant with a strong R^2 of 0.826, which can be directly acted upon. Servers may make a reasonable projection of their earnings per shift, and management can determine a realistic tipping policy or incentive program. A prediction such as this that a \$45 dinner bill produces \$7.59 in tips versus the \$3.24 expected for an \$18.50 lunch bill is the type of concrete and quantifiable information that makes a machine learning model a business tool.

9. Conclusion

This project showed how predictive analytics can be implemented in three widely disparate domains throughout the entire lifecycle. We observed that deep learning is effective when the phenomenon is sequential, like stock prices, classical time-series models like Holt-Winters can be powerful and interpretable in the context of retail sales data that has a clear seasonality pattern, and simple linear regression can be more accurate than more sophisticated models when the underlying relationship is already strong, as was true for tipping. We focused specifically on time-based splitting for temporal data, explained the relationship between model complexity and interpretability, and created one model and packaged it up into a production-ready pipeline. The entire journey from raw data to business insight is captured in the final products – a well-documented GitHub repository, a reproducible Jupyter notebook, and this report.

Github Repository Link:

<https://github.com/sourav1903078/Predictive-Analytics-Project>